

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



SISTEMAS DE INTELIGENCIA DE NEGOCIOS SI807-U
EXAMEN SUSTITUTORIO

DOCENTE:

Dr. Ing. Hilario Aradiel Castañeda

ALUMNO:

Código UNI	Apellidos y Nombres	Rama Examen Sustitutorio
20212541J	Cabana Cazani Gabriel	ExSusti_Cabana_Cazani_20212541J

Contenido

1. RESUMEN EJECUTIVO	4
2. CONTEXTO DEL NEGOCIO Y PLANTEAMIENTO DEL PROBLEMA	4
2.1 Contexto del Negocio	4
2.2 Planteamiento del Problema	5
3. OBJETIVOS DE LA SOLUCIÓN DE INTELIGENCIA DE NEGOCIOS	6
3.1. Objetivo General	6
3.2. Objetivos Específicos	6
4. ANÁLISIS DEL DATASET (FUENTE OFICIAL)	6
4.1. Descripción de la fuente de datos	6
4.2. Estructura y campos relevantes	7
4.3. Volumen y período de la información	7
4.4. Calidad de datos y hallazgos	8
5. ARQUITECTURA DE LA SOLUCIÓN BI (LOCAL)	9
5.1 Diagrama de arquitectura	9
5.2 Flujo end-to-end de datos	9
5.3 Herramientas y tecnologías utilizadas	10
6. PROCESO ETL / ELT	11
6.1 Estrategia de ingesta	11
6.2 Transformaciones aplicadas	12
6.3 Reglas de calidad de datos	12
6.4 Evidencia de ejecución	13
7. MODELO DIMENSIONAL	16
7.1 Diseño del modelo analítico	16
7.2 Tabla(s) de hechos	16
7.3 Dimensiones	17
7.4 Definición del grano	17
8. DEFINICIÓN DE KPIs	18
8.1 KPIs académicos	18
8.2 KPIs de gestión	18
8.3 Fórmulas y fuentes de datos	19
9. DASHBOARDS Y VISUALIZACIÓN	21
9.1 Dashboards desarrollados	21

9.2 KPIs visualizados	22
9.3 Filtros y capacidades de análisis	23
9.4 Evidencia de despliegue local	23
10. REPRODUCIBILIDAD DE LA SOLUCIÓN	25
11. RESULTADOS Y ANÁLISIS	26
11.1. Análisis de Gestión y Demanda (Operativo)	33
11.2. Análisis Académico (Calidad y Perfil)	34
11.3. Síntesis de Resultados	35
12. CONCLUSIONES	35
13. LIMITACIONES Y MEJORAS FUTURAS	36
13.1. Limitaciones de la Solución Actual	36
13.2. Mejoras Futuras y Trabajo Futuro	36
14. REFERENCIAS	37
ANEXOS	37

1. RESUMEN EJECUTIVO

El presente proyecto detalla el diseño e implementación de una solución integral de Inteligencia de Negocios y Big Data orientada a modernizar el análisis estratégico del proceso de admisión del CEPRE-UNI. Ante el desafío de gestionar grandes volúmenes de información histórica dispersa, se desplegó una arquitectura escalable sobre el ecosistema **Hadoop**, utilizando **Apache Spark (PySpark)** para ejecutar procesos automatizados de extracción, transformación y carga (ETL) que aseguran la limpieza, deduplicación y calidad de los datos. Esta información depurada fue consolidada en un **Data Warehouse** bajo un modelo dimensional (Esquema Estrella) en **Apache Hive**, optimizado para responder consultas analíticas complejas con rapidez. Como resultado final, se integró la plataforma **Microsoft Power BI** para desarrollar dos tableros de control interactivos: un **Dashboard de Gestión**, que permite a las autoridades monitorear la evolución de la demanda, la competitividad por vacante y la efectividad de las modalidades de ingreso; y un **Dashboard Académico**, enfocado en evaluar la calidad educativa mediante indicadores de selectividad, rendimiento promedio y perfiles demográficos de los ingresantes, dotando así a la institución de una herramienta robusta para la toma de decisiones basada en evidencia.

2. CONTEXTO DEL NEGOCIO Y PLANTEAMIENTO DEL PROBLEMA

2.1 Contexto del Negocio

Institución: Centro Preuniversitario de la Universidad Nacional de Ingeniería (CEPRE-UNI).

Descripción de la Organización: El CEPRE-UNI es la institución encargada de la preparación académica y selección de postulantes para el ingreso directo a la Universidad Nacional de Ingeniería (UNI), una de las casas de estudio más prestigiosas y exigentes del país. Su modelo de negocio se basa en la captación de miles de estudiantes por ciclo (Ordinario, Intensivo, Escolar), quienes rinden evaluaciones periódicas para alcanzar una vacante en las distintas facultades de ingeniería y arquitectura.

Procesos Críticos: El ciclo de vida de la información en CEPRE-UNI abarca desde la inscripción del postulante (registro de datos demográficos, socioeconómicos y colegio de procedencia) hasta la publicación de resultados (notas, asistencia y cuadro de mérito). La toma de decisiones en esta institución depende de dos pilares:

1. **Gestión Administrativa:** Evaluar la demanda, la rentabilidad por ciclo y la efectividad de las campañas de marketing en diferentes regiones del país.
2. **Gestión Académica:** Monitorear el nivel de dificultad de los exámenes, la tasa de éxito de los estudiantes y la calidad educativa impartida.

2.2 Planteamiento del Problema

Situación Actual: A pesar de generar grandes volúmenes de datos transaccionales en cada proceso de admisión, la institución carecía de una arquitectura centralizada para la explotación analítica de esta información. Los datos históricos de los postulantes se encontraban dispersos en un archivo plano (CSV), hojas de cálculo y bases de datos aisladas, lo que generaba silos de información desconectados.

Problemas Identificados:

1. **Dispersión y Calidad del Dato:** La falta de un repositorio unificado provocaba inconsistencias. Existían registros duplicados y falta de estandarización en campos clave como "Colegios" o "Ubicación Geográfica", lo que dificultaba obtener una visión única del postulante.
2. **Incapacidad de Análisis Histórico:** Los reportes existentes eran estáticos y se limitaban al ciclo actual. No existía una herramienta que permitiera comparar tendencias a lo largo del tiempo (ej. *"¿Ha bajado la calidad de los ingresantes respecto a hace 5 años?"* o *"¿Está creciendo la demanda en provincias?"*).
3. **Toma de Decisiones Reactiva:** Al no contar con indicadores de gestión automatizados, las autoridades tomaban decisiones basadas en la intuición o en reportes manuales que tardaban días en generarse, perdiendo oportunidades estratégicas para mejorar la captación o ajustar la exigencia académica.
4. **Procesos Manuales y Error Humano:** La consolidación de notas y estadísticas se realizaba mediante procesos manuales propensos al error, lo que ponía en riesgo la fiabilidad de los resultados presentados a la dirección.

Necesidad de la Solución: Ante este escenario, resultaba imperativo implementar una solución de **Inteligencia de Negocios (BI) y Big Data** capaz de procesar, limpiar y visualizar la información histórica, transformando los datos crudos en conocimiento accionable para la mejora continua de la institución.

3. OBJETIVOS DE LA SOLUCIÓN DE INTELIGENCIA DE NEGOCIOS

3.1. Objetivo General

Diseñar e implementar una solución integral de Inteligencia de Negocios y Big Data para el CEPRE-UNI, que permita centralizar, procesar y visualizar la información histórica de los procesos de admisión, facilitando a la dirección la toma de decisiones estratégicas orientadas a la optimización de la gestión administrativa y el aseguramiento de la calidad académica.

3.2. Objetivos Específicos

1. **Centralización y Calidad de Datos (Capa Técnica):** Implementar un flujo automatizado de Extracción, Transformación y Carga (ETL) utilizando **Apache Spark**, con el fin de integrar fuentes de datos dispersas, eliminar duplicidad de registros y estandarizar la información de postulantes para asegurar una "fuente única de verdad".
2. **Modelado para el Análisis (Capa de Datos):** Construir un Data Warehouse bajo un modelo dimensional (Esquema Estrella) en **Apache Hive**, estructurando la información en tablas de hechos y dimensiones que optimicen los tiempos de respuesta ante consultas analíticas complejas.
3. **Monitoreo de la Gestión Administrativa (Capa de Negocio):** Desarrollar un **Dashboard de Gestión** en Microsoft Power BI que permita visualizar indicadores clave (KPIs) sobre la evolución de la demanda, el análisis de modalidades de ingreso y la competitividad (ratio postulantes/vacante) para mejorar las estrategias de captación.
4. **Evaluación del Rendimiento Educativo (Capa Académica):** Implementar un **Dashboard Académico** que analice la selectividad del examen y el perfil del ingresante, permitiendo identificar patrones de rendimiento según el colegio de procedencia, género y ubicación geográfica.
5. **Soporte a la Toma de Decisiones:** Proveer a las autoridades de la institución de herramientas visuales e interactivas que sustituyan los reportes manuales estáticos, permitiendo realizar análisis comparativos históricos y segmentaciones dinámicas en tiempo real.

4. ANÁLISIS DEL DATASET (FUENTE OFICIAL)

4.1. Descripción de la fuente de datos

La fuente de información principal para el presente proyecto corresponde al histórico transaccional de los procesos de admisión del CEPRE-UNI. Se trata de **datos estructurados** generados por los sistemas operativos de registro e inscripción de la institución.

El dataset original fue extraído en formato de archivo plano (**CSV - Comma Separated Values**) para facilitar su ingesta en el Data Lake. Esta fuente contiene la información detallada de los postulantes, sus antecedentes académicos (colegio de procedencia), datos sociodemográficos y los resultados obtenidos en las evaluaciones, constituyendo el insumo base (*Raw Data*) para el proceso de transformación ETL.

4.2. Estructura y campos relevantes

El archivo presenta una estructura tabular compuesta por filas y columnas, donde cada columna representa un atributo del postulante o del proceso. Los campos más relevantes identificados para el modelado dimensional y la generación de indicadores son:

Tipo de Dato	Campo (Atributo)	Descripción Funcional
Dimensión	id_postulante	Identificador único anonimizado del estudiante.
Dimensión	anio	Periodo académico del proceso de admisión.
Dimensión	carrera	Especialidad a la que postula (Ej: Ingeniería Civil).
Dimensión	modalidad	Canal de ingreso (Ordinario, Traslado, etc.).
Dimensión	departamento	Ubicación geográfica de residencia del postulante.
Dimensión	nombre_colegio	Institución educativa de origen (EBR).
Dimensión	sexo	Género del postulante (M/F).
Métrica	puntaje_final	Nota vigesimal obtenida en el examen.
Métrica	ingreso_flag	Indicador binario (1/0) derivado que señala si obtuvo vacante.
Métrica	edad	Edad del estudiante al momento de postular.

4.3. Volumen y período de la información

- **Volumetría:** El dataset consolidado consta de un total de **8,470 registros** procesados.

- **Granularidad:** La unidad mínima de información corresponde a **un registro por postulante por proceso de admisión**.
- **Período:** La información abarca un histórico de múltiples periodos académicos, lo que permite realizar análisis de tendencias temporales y evaluar la evolución de la demanda y el rendimiento a lo largo de los distintos ciclos de admisión.

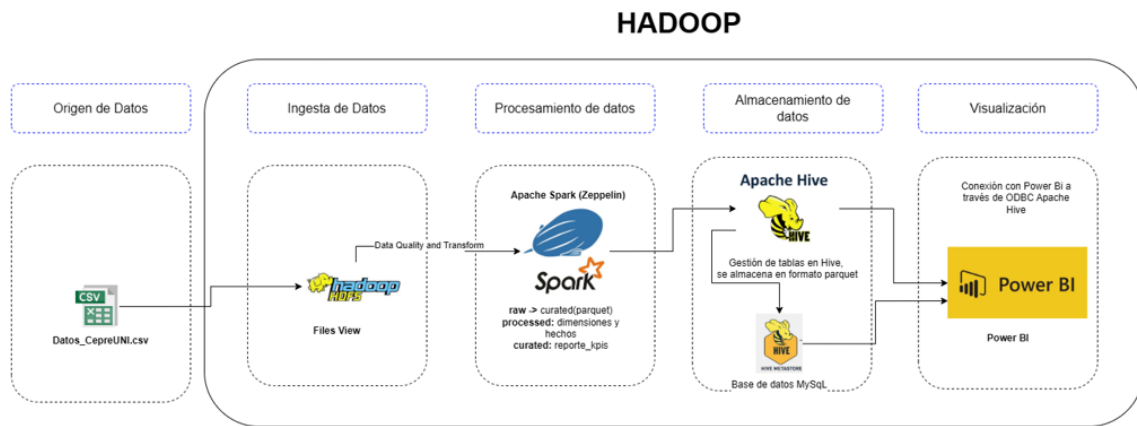
4.4. Calidad de datos y hallazgos

Durante la fase de exploración y diagnóstico inicial (*Data Profiling*), se evaluó la calidad del dataset crudo, identificando los siguientes hallazgos que requirieron tratamiento mediante **Apache Spark**:

1. **Duplicidad de Registros:** Se detectaron inconsistencias generadas por la extracción reiterada de datos, lo que ocasionaba una inflación en las métricas (problema de "explosión de datos"). Esto fue corregido mediante procesos de deduplicación (*distinct*) en la capa de procesamiento.
2. **Inconsistencia en Cadenas de Texto:** Atributos categóricos como departamento y nombre_colegio presentaban variaciones de escritura (uso de mayúsculas/minúsculas y tildes), lo que impedía una agrupación correcta. Se aplicaron funciones de normalización y limpieza (*trim, upper*).
3. **Tipos de Datos (Casting):** Campos numéricos críticos, como el puntaje_final y la edad, se encontraban originalmente en formato texto (*string*), requiriendo una conversión explícita (*casting*) a tipos numéricos (*double/int*) para permitir el cálculo de promedios y agregaciones estadísticas.
4. **Valores Nulos:** Se identificaron registros con valores nulos en campos no críticos, los cuales fueron tratados mediante reglas de imputación o exclusión según su impacto en el modelo de negocio.

5. ARQUITECTURA DE LA SOLUCIÓN BI (LOCAL)

5.1 Diagrama de arquitectura



La solución propuesta sigue una arquitectura de referencia moderna para Big Data conocida como **Arquitectura por Capas** (o Arquitectura Medallion), desplegada sobre un clúster de **Hadoop**. Esta estructura garantiza el desacoplamiento entre el almacenamiento, el procesamiento y la visualización.

Componentes del Diagrama (Guía para tu diseño):

1. **Fuentes de Datos (Source):** Archivos planos (CSV) de los sistemas transaccionales del CEPRE-UNI.
2. **Capa de Ingesta y Almacenamiento (Data Lake - HDFS):** Repositorio centralizado donde residen los datos crudos y procesados.
3. **Capa de Procesamiento (Compute Engine):** Apache Spark actúa como el motor de transformación, ejecutando limpieza y validación en memoria.
4. **Capa de Consultas y Almacén (Data Warehouse):** Apache Hive estructura los datos limpios en tablas relacionales (Hechos y Dimensiones).
5. **Capa de Presentación (Frontend):** Microsoft Power BI se conecta al almacén para la visualización de Dashboards.

5.2 Flujo end-to-end de datos

El flujo de datos describe el ciclo de vida de la información, desde su extracción hasta su consumo final. El proceso se divide en cuatro etapas secuenciales:

1. **Ingesta (Extraction):**

- Los datos históricos de admisión son extraídos en formato .csv.
- Se cargan al sistema de archivos distribuido (**HDFS**) en una carpeta designada como *Landing Zone* o *Raw Zone*.

2. Transformación y Limpieza (Transformation - Spark):

- Se inicia una sesión de **SparkSession** (PySpark).
- **Lectura:** Se leen los archivos crudos creando DataFrames iniciales.
- **Limpieza:** Se ejecutan transformaciones críticas: eliminación de registros duplicados (*dropDuplicates*), casting de tipos de datos (convertir cadenas a enteros/decimales) y estandarización de textos (mayúsculas, eliminación de espacios).
- **Generación de IDs:** Se crean llaves subrogadas para las dimensiones (ej. id_carrera, id_tiempo).

3. Carga y Modelado (Load - Hive):

- Los DataFrames limpios se escriben en **Apache Hive** en formato **Parquet** (columnar y comprimido) para optimizar el rendimiento.
- Se materializa el **Esquema Estrella**, separando la información en una tabla de hechos (fact_admision) y tablas de dimensiones (dim_carrera, dim_tiempo, dim_geografía, etc.).

4. Visualización (Consumption - Power BI):

- Power BI establece conexión con Hive mediante el conector ODBC/DirectQuery.
- Se importan las tablas dimensionales y de hechos.
- Se definen las relaciones en el modelo semántico y se calculan las medidas DAX para los reportes de Gestión y Académico.

5.3 Herramientas y tecnologías utilizadas

Para la implementación de la solución se seleccionó un *stack* tecnológico de código abierto (Open Source) complementado con herramientas líderes de visualización.

Tecnología	Rol en la Arquitectura	Justificación de su uso
Apache Hadoop (HDFS)	Almacenamiento (Storage)	Provee un sistema de archivos distribuido, escalable y tolerante a fallos, ideal para almacenar el histórico de datos sin límite de espacio.
Apache Spark (PySpark)	Procesamiento (ETL)	Motor de procesamiento en memoria. Se eligió por ser hasta 100 veces más rápido que MapReduce tradicional, permitiendo limpiar y transformar los 8,470 registros en segundos.
Apache Hive	Data Warehouse (SQL)	Facilita la proyección de estructura (esquemas) sobre los datos almacenados en HDFS, permitiendo realizar consultas tipo SQL y servir como interfaz para Power BI.
Apache Zeppelin	Interfaz de Desarrollo	Cuaderno interactivo (Notebook) utilizado para escribir, documentar y ejecutar el código Python/SQL de manera exploratoria.
Microsoft Power BI	Visualización (BI)	Herramienta líder de Business Intelligence. Se seleccionó por su capacidad de integración, interactividad y potencia para modelado de datos (DAX).

6. PROCESO ETL / ELT

6.1 Estrategia de ingesta

Para la incorporación de los datos al Data Lake, se definió una estrategia de **procesamiento por lotes (Batch Processing)**, dado que la naturaleza de los datos de admisión es histórica y no requiere transmisión en tiempo real.

El flujo de ingesta se diseñó de la siguiente manera:

1. **Zona de Aterrizaje (Landing Zone):** Los archivos planos (.csv) conteniendo la data histórica fueron cargados al sistema de archivos distribuido (**HDFS**) en la ruta /user/alumno/Examen_Sustitutorio/data/raw.
2. **Lectura con Spark:** Se utilizó **Apache Spark (PySpark)** para la lectura de estos archivos, infiriendo el esquema inicial (inferSchema=True) y cargándolos en memoria mediante DataFrames para su procesamiento distribuido.

6.2 Transformaciones aplicadas

El núcleo del procesamiento se realizó mediante scripts de PySpark en Apache Zeppelin. Las transformaciones clave para convertir la data cruda en un Modelo Dimensional (Esquema Estrella) fueron:

1. Limpieza y Estandarización:

- **Normalización de Textos:** Se aplicaron funciones `trim()` y `upper()` a campos categóricos como `nombre_colegio`, `carrera` y `departamento` para eliminar espacios en blanco y unificar mayúsculas, evitando duplicados por sintaxis.
- **Casteo de Tipos (Type Casting):** Conversión de campos numéricos que llegaron como texto. Se transformó `puntaje_final` a `Decimal/Double` y `edad` a `Integer` para permitir cálculos matemáticos.

2. Generación de Dimensiones (Tablas Maestras):

Se extrajeron los valores únicos de los atributos descriptivos para crear las tablas dimensionales. A cada una se le asignó un índice subrogado mediante la función `monotonically_increasing_id()` o `row_number()`.

- *Dimensiones creadas:* `dim_carrera`, `dim_tiempo`, `dim_modalidad`, `dim_geografia`, `dim_institucion`, `dim_candidato`.

3. Construcción de la Tabla de Hechos (Fact Table):

Se realizó el cruce (*Join*) entre el dataset principal y las dimensiones creadas previamente. Se sustituyeron los campos descriptivos (ej. nombre de la carrera) por sus respectivas llaves foráneas (`id_carrera`), resultando en la tabla `fact_admision` optimizada para el análisis.

6.3 Reglas de calidad de datos

Para garantizar la fiabilidad de la información, se implementaron reglas de validación lógica durante la ejecución del script:

1. **Unicidad (Deduplicación):** Se identificó un problema crítico de duplicidad masiva ("explosión de datos") debido a cargas previas erróneas. Se aplicó la regla de `distinct()` sobre el dataset total y las dimensiones, reduciendo el volumen de registros erróneos (aprox. 1.8 millones) a la cifra correcta (**8,470 registros únicos**).
2. **Integridad Referencial:** Se aseguró que todos los registros en la tabla de hechos tuvieran correspondencia en las tablas dimensionales, evitando registros "huérfanos" que romperían el modelo en Power BI.
3. **Consistencia de Dominio:** Validación de que los campos críticos como `puntaje_final` se encuentren dentro de los rangos permitidos (0 a 20) y que no existan valores nulos (NULL) en las llaves primarias.

6.4 Evidencia de ejecución

Ejecución del Script en Zeppelin (Estado FINISHED)

```
[spark2.pyspark
from pyspark.sql import functions as f
from pyspark.sql.window import Window

print("--- BLOQUE 1: LECTURA Y LIMPIEZA ---")

# 1. Configuración para Hive
spark.conf.set("hive.exec.dynamic.partition", "true")
spark.conf.set("hive.exec.dynamic.partition.mode", "nonstrict")

# 2. Lectura del CSV (Ruta 202125411)
ruta_csv = "/user/almoro/Examen/Institutorio/Cabana_Cazani_202125411/data/raw"
df_raw = spark.read.option("header", "true").csv(ruta_csv)

# 3. Limpieza de Textos (Mayúsculas y espacios)
cols_texto = ["ESPECIALIDAD", "MODO_INGRESO", "SEXO", "COLEGIO",
              "DOMICILIO_DEPA", "DOMICILIO_PROV", "DOMICILIO_DIST",
              "COLEGIO_DEPA", "COLEGIO_PROV", "COLEGIO_DIST"]

df_limpio = df_raw
for c in cols_texto:
    df_limpio = df_limpio.withColumn(c, f.upper(f.trim(f.col(c))))

# 4. Reemplazar Nulos, Tipos de Datos y Columnas Nuevas
df_limpio = df_limpio.fillna({"CALIF_FINAL": "0.0", "INGRESO": "NO"}) \
    .withColumn("ANIO_POSTULA", f.col("ANIO_POSTULA").cast("int")) \
    .withColumn("CICLO_POSTULA", f.col("CICLO_POSTULA").cast("int")) \
    .withColumn("ANIO_NACIMIENTO", f.col("ANIO_NACIMIENTO").cast("int")) \
    .withColumn("COLEGIO_ANIO_INGRESO", f.col("COLEGIO_ANIO_INGRESO").cast("int")) \
    .withColumn("CALIF_FINAL", f.col("CALIF_FINAL").cast("double")) \
    .withColumn("id_tiempo", f.concat(f.col("ANIO_POSTULA"), f.col("CICLO_POSTULA")).cast("int")) \
    .withColumn("edad_calculada", (f.col("ANIO_INGRESO") - f.col("ANIO_NACIMIENTO")).cast("int")) \
    .withColumn("geo_domicilio", f.concat_ws("-", "DOMICILIO_DEPA", "DOMICILIO_PROV", "DOMICILIO_DIST")) \
    .withColumn("geo_colegio", f.concat_ws("-", "COLEGIO_DEPA", "COLEGIO_PROV", "COLEGIO_DIST")) \
    .withColumn("flag_ingreso", f.when(f.col("INGRESO") == "SI", 1).otherwise(0))

# 5. Guardar en Memoria y Vista Temporal (Clave para compartir entre bloques)
df_limpio.cache()
df_limpio.createOrReplaceTempView("tmp_master_limpio")

%%python3
# Bloque 1 Completado. Registros listos: 8470

--- BLOQUE 1: LECTURA Y LIMPIEZA ---
✓ Bloque 1 Completado. Registros listos: 8470
```

```
[spark2.pyspark
# Recuperamos librerías y data del paso anterior
from pyspark.sql import functions as f
from pyspark.sql.window import Window
df = spark.table("tmp_master_limpio")

print("--- BLOQUE 2: CARGA DE DIMENSIONES ---")

# 1. Carrera
print("-> Procesando Carrera...")
w = Window.orderBy("ESPECIALIDAD")
df.select("ESPECIALIDAD").distinct().withColumn("id", f.row_number().over(w)) \
    .select(f.col("id"), f.col("ESPECIALIDAD")).alias("nombre_carrera") \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_carrera")

# 2. Modalidad
print("-> Procesando Modalidad...")
w = Window.orderBy("MODO_INGRESO")
df.select("MODO_INGRESO").distinct().withColumn("id", f.row_number().over(w)) \
    .select(f.col("id"), f.col("MODO_INGRESO")).alias("descripcion_modalidad") \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_modalidad")

# 3. Tiempo
print("-> Procesando Tiempo...")
df.select("id_tiempo", "ANIO_POSTULA", "CICLO_POSTULA").distinct() \
    .withColumn("desc", f.concat(f.lit("Ciclo "), f.col("ANIO_POSTULA"), f.lit("-"), f.col("CICLO_POSTULA"))) \
    .select("id_tiempo", "ANIO_POSTULA", "CICLO_POSTULA", "desc") \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_tiempo")

# 4. Geografía
print("-> Procesando Geografía...")
g1 = df.select(f.col("geo_domicilio").alias("id"), "DOMICILIO_DEPA", "DOMICILIO_PROV", "DOMICILIO_DIST")
g2 = df.select(f.col("geo_colegio").alias("id"), "COLEGIO_DEPA", "COLEGIO_PROV", "COLEGIO_DIST")
g1.union(g2).distinct().select("id", "DOMICILIO_DEPA", "DOMICILIO_PROV", "DOMICILIO_DIST", f.lit("PENU")) \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_geografia")

# 5. Institución
print("-> Procesando Institución...")
w = Window.orderBy("COLEGIO")
df.select("COLEGIO", "geo_colegio").distinct().withColumn("id", f.row_number().over(w)) \
    .select(f.col("id"), f.col("COLEGIO").alias("nombre_colegio"), f.col("geo_colegio").alias("id_geo_colegio")) \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_institucion")

# 6. Candidato
print("-> Procesando Candidato...")
df.select("TOMADO", "SEXO", "ANIO_NACIMIENTO", "NACIMIENTO_PAIS").distinct() \
    .select(f.col("IDPAIS").alias("id_país"), f.col("SEXO"), f.col("ANIO_NACIMIENTO"), f.col("NACIMIENTO_PAIS").alias("país_nacimiento")) \
    .write.mode("overwrite").insertInto("db_cepreuni.dim_candidato")

print("✓ Bloque 2 Completado: Dimensiones cargadas.")
```

```
--- BLOQUE 2: CARGA DE DIMENSIONES ---
-> Procesando Carrera...
-> Procesando Modalidad...
-> Procesando Tiempo...
-> Procesando Geografía...
-> Procesando Institución...
-> Procesando Candidato...
✓ Bloque 2 Completado: Dimensiones cargadas.

Took 1 min 31 sec. Last updated by anonymous at December 21 2025, 2:01:11 PM.
```

```
[spark2.pyspark
from pyspark.sql import functions as f
df = spark.table("tmp_master_limpio")

print("--- BLOQUE 3: CARGA FINAL DE HECHOS ---")

# 1. Lemnos las dimensiones para obtener los IDs
d_car = spark.table("db_cepreuni.dim_carrera")
d_mod = spark.table("db_cepreuni.dim_modalidad")
d_ins = spark.table("db_cepreuni.dim_institucion")

# 2. Hacemos el Join (Cruc)
df_final = df.join(d_car, df.ESPECIALIDAD == d_car.nombre_carrera, "left") \
    .join(d_mod, df.MODO_INGRESO == d_mod.descripcion_modalidad, "left") \
    .join(d_ins, (df.COLEGIO == d_ins.nombre_colegio) & (df.geo_colegio == d_ins.id_geo_colegio), "left") \
    .select(
        df.id_tiempo,
        df.TOMADO.alias("id_candidato"),
        d_car.id_carrera,
        d_ins.id_colegio,
        d_mod.id_modalidad,
        df.geo_domicilio.alias("id_geo_residencia"),
        df.edad_calculada.alias("edad_postulacion"),
        (df.ANIO_POSTULA - df.COLEGIO_ANIO_INGRESO).alias("años_desde_egreso"),
        df.CALIF_FINAL.alias("puntaje_final"),
        df.flag_ingreso.alias("ingreso_flag"),
        f.lit(3).alias("cantidad"),
        df.ANIO_POSTULA.alias("año")
    )

# 3. Escritura Final
print("Escribiendo datos en fact_admisión...")
df_final.write.mode("overwrite").insertInto("db_cepreuni.fact_admisión")

print("🎉 ¡PROYECTO TERMINADO! Base de datos db_cepreuni poblada.")

--- BLOQUE 3: CARGA FINAL DE HECHOS ---
Escribiendo datos en fact_admisión...
🎉 ¡PROYECTO TERMINADO! Base de datos db_cepreuni poblada.

Took 3 sec. Last updated by anonymous at December 21 2025, 2:01:16 PM.
```

Descripción: Se observa la ejecución exitosa de los párrafos de código PySpark encargados de la transformación y carga.

Validación de Conteo de Registros (Data Quality)

spark2.sql

SELECT
t.anio,
c.nombre_carrera,
COUNT(*) as total_postulantes,
SUM(f.ingreso_flag) as total_ingresantes,
ROUND((SUM(f.ingreso_flag) / COUNT(*)) * 100, 2) as tasa_ingreso_pct
FROM db_ceprenu1.fact_admision f
JOIN db_ceprenu1.dim_tiempo t ON f.id_tiempo = t.id_tiempo
JOIN db_ceprenu1.dim_carrera c ON f.id_carrera = c.id_carrera
GROUP BY t.anio, t.nombre_carrera
ORDER BY total_postulantes DESC
LIMIT 10

FINISHED

anio	nombre_carrera	total_postulantes	total_ingresantes	tasa_ingreso_pct
2024	SIN ESPECIALIDAD	4753	0	0.0
2025	SIN ESPECIALIDAD	2149	0	0.0
2024	INGENIERÍA CIVIL	105	104	99.05
2024	INGENIERÍA INDUSTRIAL	68	68	100.0
2024	INGENIERÍA DE SISTEMAS	59	59	100.0
2024	ARQUITECTURA	58	58	100.0
2024	INGENIERÍA QUÍMICA	52	52	100.0
2024	INGENIERÍA SANITARIA	47	47	100.0

Task 5 sec. Last updated by anonymous at December 21 2025, 2:01:28 PM.

spark2.sql

-- KPI GESTIÓN 1: Demanda (Volumen)
COUNT(*) as total_postulantes,

-- KPI GESTIÓN 2: Competitividad (Postulantes por cada vacante)
ROUND(COUNT(*) / SUM(f.ingreso_flag), 1) as postulantes_por_vacante,

-- KPI ACADÉMICO 1: Promedio de Notas (Nivel académico)
ROUND(AVG(f.puntaje_final), 2) as promedio_nota_general,

-- KPI ACADÉMICO 2: Tasa de Ingreso (% de éxito)
ROUND((SUM(f.ingreso_flag) / COUNT(*)) * 100, 2) as tasa_ingreso_pct
FROM db_ceprenu1.fact_admision f
JOIN db_ceprenu1.dim_tiempo t ON f.id_tiempo = t.id_tiempo
GROUP BY t.anio

FINISHED

anio	total_postulantes	postulantes_por_vacante	promedio_nota_general	tasa_ingreso_pct
2025	2677	5.1	7.75	19.72
2024	5793	5.6	7.26	17.94

Descripción: Consulta de validación SQL que confirma la reducción de duplicados, obteniendo el total exacto de 8,470 postulantes únicos tras la limpieza.

Tablas creadas en Hive (Data Warehouse)

Database Explorer

db_ceprenu1

Search tables...

Databases

db ceprenu1

cur reporte carreras

cur reporte colegios

dim candidato

dim carrera

dim geografia

dim institucion

dim modalidad

dim tiempo

fact admision

raw postulantes

default

foodmart

Query Editor

Worksheet

Worksheet (1)

Worksheet (2) *

1

USE db_ceprenu1;

2

3

-- 1. TABLA REPORTE POR CARRERA (KPIs de Ingreso por Especialidad)

4

-- Ubicación: ../data/curated/reporte_carreras

5

CREATE EXTERNAL TABLE cur_reporte_carreras (

6

anio INT,

7

nombre_carrera STRING,

8

total_postulantes INT,

9

total_ingresantes INT,

10

tasa_ingreso_pct DOUBLE

11

)

12

STORED AS PARQUET

13

LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_202125417/data/curated/reporte_carreras';

14

15

-- 2. TABLA REPORTE POR COLEGIO (Ranking de Colegios)

16

-- Ubicación: ../data/curated/reporte_colegios

17

CREATE EXTERNAL TABLE cur_reporte_colegios (

18

nombre_colegio STRING,

19

departamento STRING,

20

total_postulantes INT,

21

total_ingresantes INT

22

)

23

STORED AS PARQUET

24

LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_202125417/data/curated/reporte_colegios';

```
Worksheet x Worksheet (1) x Worksheet (2) * x
1 DROP DATABASE IF EXISTS db_cepreni1 CASCADE;
2 CREATE DATABASE db_cepreni1;
3 USE db_cepreni1;
4
5 -- 1. TABLA RAW (Lectura del CSV)
6 CREATE EXTERNAL TABLE raw_postulantes (
7   IDHASH STRING,
8   COLEGIO STRING,
9   COLEGIO_DEPA STRING,
10  COLEGIO_PROV STRING,
11  COLEGIO_DIST STRING,
12  COLEGIO_PAIS STRING,
13  COLEGIO_ANIO_EGRESO STRING,
14  ESPECIALIDAD STRING,
15  ANIO_POSTULA INT,
16  CICLO_POSTULA INT,
17  DOMICILIO_DEPA STRING,
18  DOMICILIO_PROV STRING,
19  DOMICILIO_DIST STRING,
20  ANIO_NACIMIENTO INT,
21  NACIMIENTO_PAIS STRING,
22  NACIMIENTO_DEPA STRING,
23  NACIMIENTO_PROV STRING,
24  NACIMIENTO_DIST STRING,
25  SEXO STRING,
26  CALIF_FINAL DOUBLE,
27  INGRESO STRING,
28  MODO_INGRESO STRING,
29  FECHA_CORTE BIGINT
30 )
31 ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
32 STORED AS TEXTFILE
33 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/raw'
34 TBLPROPERTIES ("skip.header.line.count"="1");
35
36 -- 2. TABLAS DIMENSIONALES (Parquet)
37 CREATE EXTERNAL TABLE dim_carrera (id_carrera INT, nombre_carrera STRING)
38 STORED AS PARQUET
39 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_carrera';
40
41 CREATE EXTERNAL TABLE dim_modalidad (id_modalidad INT, descripcion_modalidad STRING)
42 STORED AS PARQUET
43 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_modalidad';
44
45 CREATE EXTERNAL TABLE dim_tiempo (id_tiempo INT, anio INT, ciclo INT, descripcion STRING)
46 STORED AS PARQUET
47 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_tiempo';
48
49 CREATE EXTERNAL TABLE dim_geografia (id_geo STRING, departamento STRING, provincia STRING, distrito STRING, pais STRING)
50 STORED AS PARQUET
51 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_geografia';
52
53 CREATE EXTERNAL TABLE dim_institucion (id_colegio INT, nombre_colegio STRING, id_geo_colegio STRING)
54 STORED AS PARQUET
55 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_institucion';
56
57 CREATE EXTERNAL TABLE dim_candidato (id_candidato STRING, sexo STRING, anio_nacimiento INT, pais_nacimiento STRING)
58 STORED AS PARQUET
59 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/dim_candidato';
60
61 -- 3. TABLA DE HECHOS (Parquet + Particionada)
62 CREATE EXTERNAL TABLE fact_admision (
63   id_tiempo INT,
64   id_candidato STRING,
65   id_carrera INT,
66   id_colegio INT,
67   id_modalidad INT,
68   id_geo_residencia STRING,
69   edad_postulacion INT,
70   anios_desde_egreso INT,
71   puntaje_final DOUBLE,
72   ingreso_flag INT,
73   cantidad INT
74 )
75 PARTITIONED BY (anio INT)
76 STORED AS PARQUET
77 LOCATION '/user/alumno/Examen_Sustitutorio/Cabana_Cazani_20212541J/data/processed/fact_admision';
```

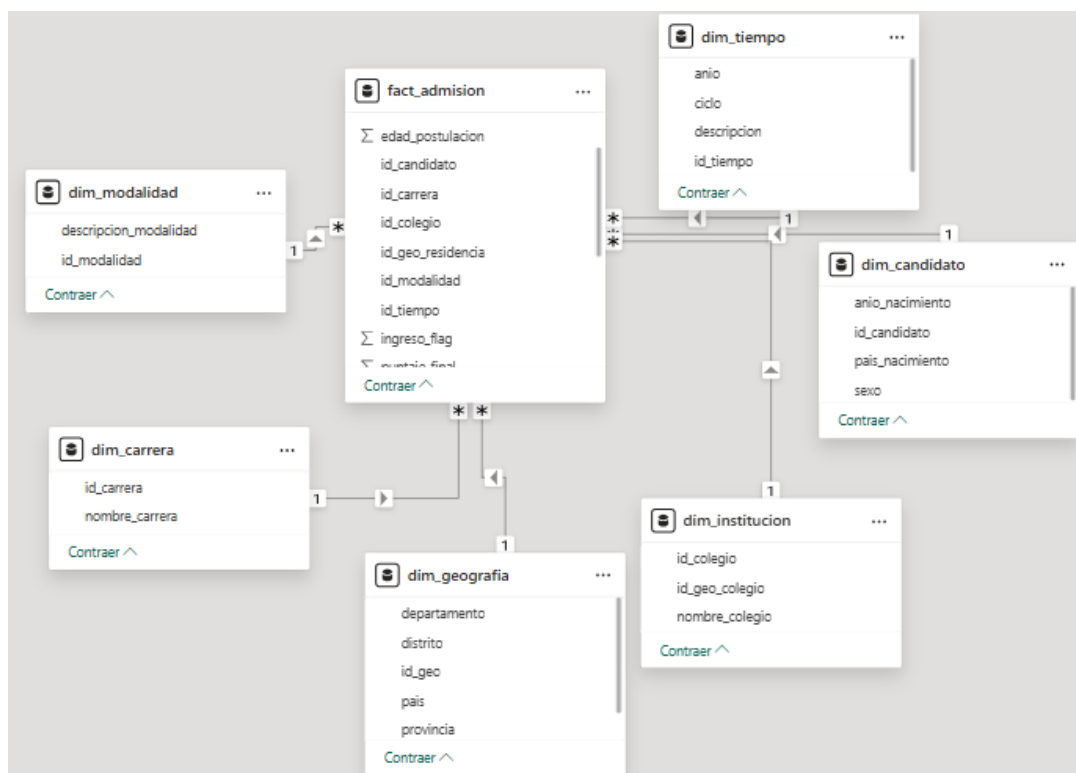
Descripción: Verificación de la persistencia de datos en el Data Warehouse (Hive), mostrando la estructura final del Esquema Estrella listo para ser consumido.

7. MODELO DIMENSIONAL

7.1 Diseño del modelo analítico

Para la estructura del Data Warehouse, se seleccionó un **Modelo Dimensional en Esquema Estrella (Star Schema)**. Esta decisión de diseño prioriza el rendimiento de las consultas analíticas y la simplicidad para el usuario final en Power BI.

A diferencia de un modelo relacional normalizado (3NF) utilizado en sistemas transaccionales, el Esquema Estrella desnormaliza los datos, centralizando las métricas numéricas en una única tabla de hechos rodeada por tablas de dimensiones descriptivas. Esta arquitectura reduce la cantidad de uniones (*joins*) complejas necesarias para generar reportes, permitiendo tiempos de respuesta rápidos al filtrar por año, carrera o modalidad.



Modelo Estrella planteado

7.2 Tabla(s) de hechos

El núcleo del modelo es la tabla de hechos denominada **fact_admision**. Esta tabla almacena los eventos transaccionales del proceso de admisión y contiene dos tipos de columnas: llaves foráneas (para conectar con las dimensiones) y medidas numéricas (para el cálculo de KPIs).

Estructura de fact_admision:

- **Llaves Foráneas (FK):** id_carrera, id_tiem po, id_modalidad, id_geo_residencia, id_institucion, id_candidato.

- **Métricas (Hechos):**

- **puntaje_final:** Nota vigesimal obtenida (Base para promedios).
- **ingreso_flag:** Valor binario (1/0) que indica si obtuvo vacante (Base para conteo de ingresantes y tasas de éxito).
- **edad_postulacion:** Edad del candidato al momento del examen.
- **anios_desde_egreso:** Tiempo transcurrido desde la culminación escolar.

7.3 Dimensiones

Las dimensiones proporcionan el contexto descriptivo ("quién", "cuándo", "dónde", "qué") necesario para filtrar y agrupar las métricas de la tabla de hechos. Se definieron seis dimensiones estratégicas:

1. **dim_tiempo:** Permite el análisis temporal.
 - *Atributos:* id_tiempo, anio, periodo (si aplica), semestre.
2. **dim_carrera:** Describe la oferta académica.
 - *Atributos:* id_carrera, nombre_carrera (Ej. Ing. Civil), facultad.
3. **dim_modalidad:** Clasifica los canales de ingreso.
 - *Atributos:* id_modalidad, descripcion_modalidad (Ej. Ordinario, Dos Primeros Puestos).
4. **dim_geografia:** Ubicación espacial del postulante.
 - *Atributos:* id_geo, departamento (Ej. LIMA, AREQUIPA).
5. **dim_institucion:** Información del colegio de procedencia.
 - *Atributos:* id_institucion, nombre_colegio, tipo_gestion (Estatual/Privado).
6. **dim_candidato:** Datos demográficos del postulante.
 - *Atributos:* id_candidato, sexo (M/F), rango_edad.

7.4 Definición del grano

La granularidad (o nivel de detalle) del modelo se definió como: **Un registro por postulante por proceso de admisión.**

- **Justificación:** Este es el nivel más atómico disponible en la fuente de datos. Mantener este grano fino, en lugar de pre-agregar los datos (ej. guardar solo totales por carrera), dota al modelo de una flexibilidad total. Permite a los usuarios de negocio "bajar" (Drill-down) desde una vista general anual hasta el detalle específico de un solo examen, recalculando indicadores dinámicamente sin perder precisión.

8. DEFINICIÓN DE KPIs

8.1 KPIs académicos

Estos indicadores permiten evaluar la selectividad del proceso y el perfil de rendimiento de los postulantes. Se visualizaron en el **Reporte Académico**:

1. Tasa de Ingreso (%):

- *Definición:* Mide el porcentaje de postulantes que logran alcanzar una vacante respecto al total de inscritos.
- *Interpretación:* Un valor del **18.50%** indica una alta selectividad, donde menos de una quinta parte logra ingresar.

2. Nota Máxima (Excelencia Académica):

- *Definición:* Representa el puntaje más alto obtenido en el proceso de admisión (escala vigesimal).
- *Interpretación:* Con un valor de **17.34**, establece el "techo" de rendimiento y referencia para el Cómputo General.

3. Porcentaje de Ausentismo:

- *Definición:* Proporción de postulantes inscritos que no se presentaron al examen o cuyo puntaje es nulo.
- *Interpretación:* Un **1.49%** refleja un bajo nivel de deserción el día de la prueba.

8.2 KPIs de gestión

Estos indicadores miden la eficiencia operativa, la demanda del mercado y la cobertura de vacantes. Se visualizaron en el **Reporte de Gestión y Demanda**:

1. Total Postulantes (Demanda):

- *Definición:* Cantidad total de personas inscritas en el proceso de admisión. Representa el alcance de la convocatoria (**8.5 mil**).

2. Total Ingresantes (Cobertura):

- *Definición:* Cantidad de alumnos que lograron una vacante efectiva. Mide el cumplimiento de la meta de ingreso (**1.6 mil**).

3. Postulantes por Vacante (Competitividad):

- *Definición:* Ratio que indica cuántas personas compiten por un solo cupo disponible.
- *Interpretación:* Un índice de **5.41** señala que por cada ingresante, hubo más de 5 competidores, evidenciando la alta demanda de las carreras ofrecidas.

4. Edad Promedio:

- *Definición:* Media aritmética de la edad de los postulantes al momento del examen.
- *Interpretación:* Con **19.48 años**, se identifica un perfil de estudiante joven, mayoritariamente recién egresado de secundaria o con 1-2 años de preparación.

8.3 Fórmulas y fuentes de datos

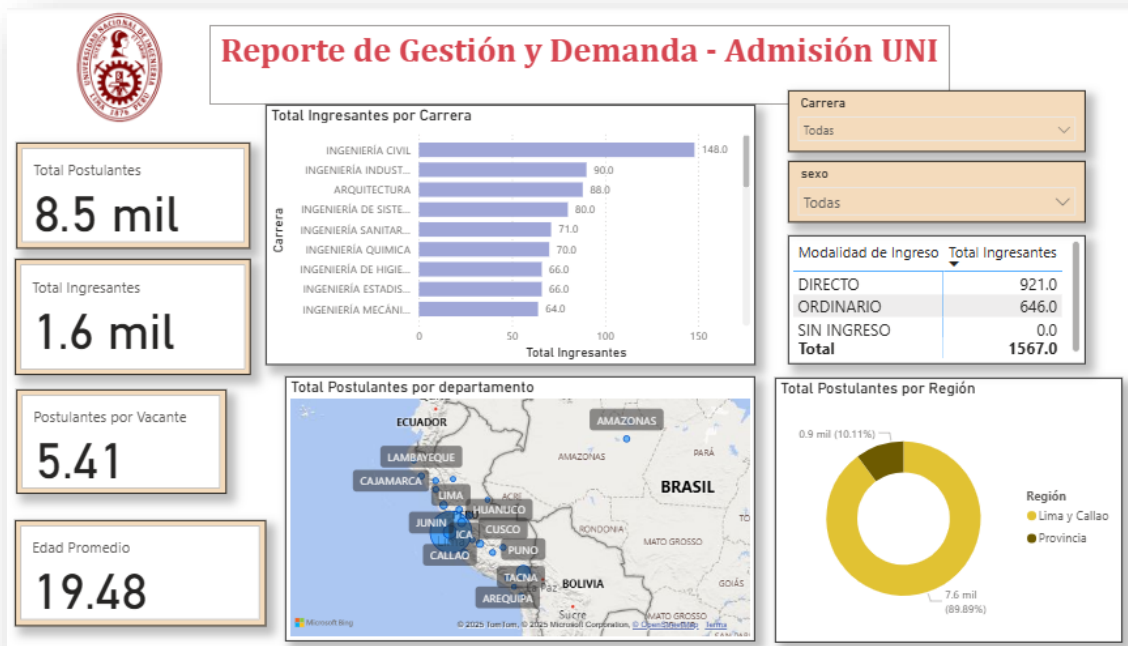
Para el cálculo de estos indicadores en Power BI, se utilizaron medidas en lenguaje **DAX (Data Analysis Expressions)**, tomando como origen la tabla de hechos fact_admision y las tablas dimensionales (dim_candidato, dim_admision).

KPI (Indicador)	Fórmula DAX Aplicada	Fuente de Datos (Columnas)
Total Postulantes	COUNTROWS (fact_admision)	Tabla: fact_admision Grano: Conteo de filas (ID único).
Total Ingresantes	CALCULATE ([Total Postulantes], fact_admision[ingreso_flag] = 1)	Tabla: fact_admision

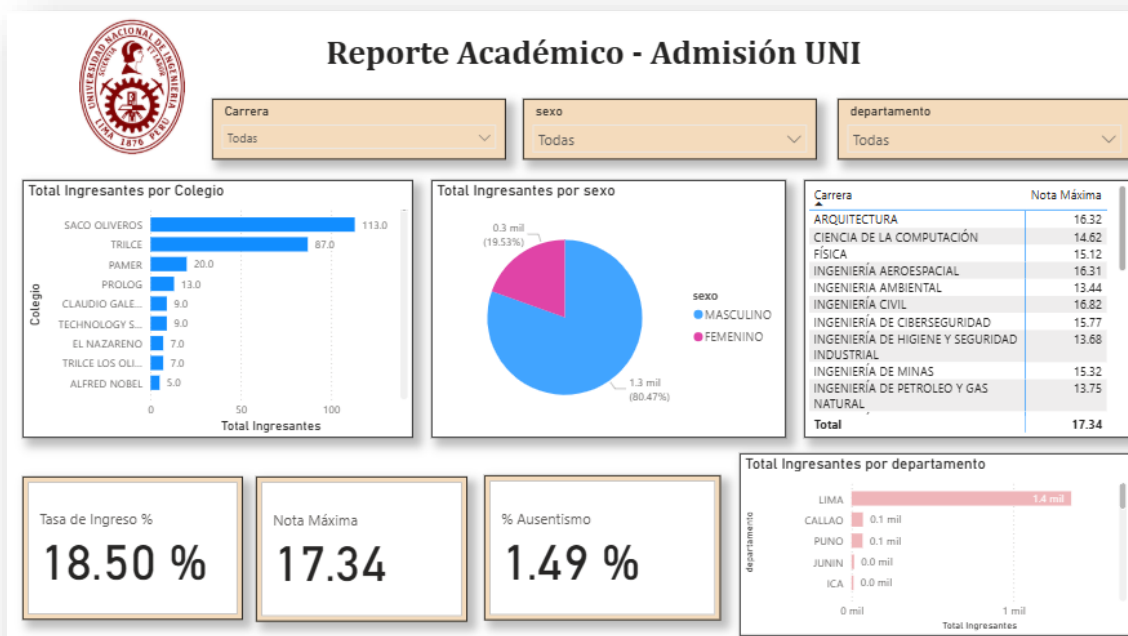
KPI (Indicador)	Fórmula DAX Aplicada	Fuente de Datos (Columnas)
		Columna: ingreso_flag (Filtro = 1).
Tasa de Ingreso %	DIVIDE([Total Ingresantes], [Total Postulantes], 0)	Medidas: [Total Ingresantes], [Total Postulantes].
Nota Máxima	MAX (fact_admision[puntaje_final])	Tabla: fact_admision Columna: puntaje_final.
% Ausentismo	DIVIDE (CALCULATE([Total Postulantes], fact_admision[puntaje_final] = 0), [Total Postulantes], 0)	Tabla: fact_admision Columna: puntaje_final (Filtro = 0).
Postulantes por Vacante	DIVIDE([Total Postulantes], [Total Ingresantes], 0)	Medidas: [Total Postulantes], [Total Ingresantes].
Edad Promedio	AVERAGE(fact_admision[edad_postulacion])	Tabla: fact_admision Columna: edad_postulacion.

9. DASHBOARDS Y VISUALIZACIÓN

9.1 Dashboards desarrollados



Dashboards1: Gestión y Demanda



Dashboards2: Reporte Académico

9.2 KPIs visualizados

La visualización de los indicadores se organizó en dos tableros estratégicos, utilizando objetos visuales nativos de Power BI para facilitar la lectura rápida de la información crítica.

A. En el Dashboard Académico: Se priorizó la evaluación de la calidad y el perfil del ingresante mediante los siguientes elementos visuales:

- **Tarjetas de KPIs (Cards):** Se dispusieron en la parte inferior para resaltar la **Tasa de Ingreso (18.50%)**, la **Nota Máxima (17.34)** y el **% de Ausentismo (1.49%)**, proporcionando un resumen ejecutivo inmediato de la selectividad.
- **Gráfico de Barras (Ranking de Colegios):** Permite visualizar las instituciones educativas con mayor cantidad de ingresantes (ej. *Saco Oliveros, Trilce*), facilitando el análisis de procedencia escolar.
- **Gráfico de Anillos (Distribución por Sexo):** Muestra la proporción demográfica de los ingresantes, evidenciando visualmente la brecha de género (80.47% Masculino vs 19.53% Femenino).
- **Tabla de Detalles:** Desglosa la nota máxima por carrera, permitiendo comparar la exigencia académica específica entre especialidades como Arquitectura e Ingeniería Civil.

B. En el Dashboard de Gestión: Se enfocó en la operatividad y la demanda del mercado:

- **Tarjetas de Volumetría:** Ubicadas a la izquierda, muestran los números absolutos de **Total Postulantes (8.5 mil)** y **Total Ingresantes (1.6 mil)**, junto con el ratio de **Postulantes por Vacante (5.41)**.
- **Mapa Geográfico (Map Chart):** Geolocaliza la procedencia de los postulantes, destacando mediante burbujas la concentración en regiones clave como Lima, Callao y Junín.
- **Gráfico de Anillos (Lima vs Provincia):** Resume la centralización de la demanda, mostrando que el 89.89% de los postulantes proviene de la capital.
- **Gráfico de Barras (Demanda por Carrera):** Ordena las especialidades según la cantidad de ingresantes, identificando rápidamente las "carreras estrella" de la institución.

9.3 Filtros y capacidades de análisis

Para dotar de interactividad a la solución, se implementaron **Segmentadores de Datos (Slicers)** y capacidades de filtrado cruzado (*Cross-filtering*), permitiendo al usuario realizar análisis dinámicos y granulares.

Segmentadores Principales: En la parte superior de ambos reportes se habilitaron tres filtros globales:

1. **Filtro de Carrera:** Permite aislar los KPIs para una especialidad específica (ej. Ver solo la tasa de ingreso de *Ingeniería de Sistemas*).
2. **Filtro de Sexo:** Facilita el análisis comparativo de rendimiento y preferencias entre hombres y mujeres.
3. **Filtro de Departamento/Región:** Permite evaluar el comportamiento de los postulantes según su zona geográfica de origen.

Capacidades Analíticas:

- **Interactividad Total:** Al seleccionar un elemento en un gráfico (por ejemplo, hacer clic en la barra de "Ingeniería Civil"), todos los demás gráficos y tarjetas se recalculan automáticamente para mostrar datos exclusivos de esa selección.
- **Navegación Drill-Down:** La estructura del modelo permite profundizar desde una vista general (toda la universidad) hasta el detalle específico (un colegio o modalidad particular).

9.4 Evidencia de despliegue local

La solución de Inteligencia de Negocios fue desplegada exitosamente en un entorno local, verificando la conexión estable entre el Data Warehouse y la herramienta de visualización.

Entorno de Ejecución:

- **Servidor de Datos:** Clúster Hadoop/Hive ejecutándose en máquina virtual local.
- **Cliente de Visualización:** Microsoft Power BI Desktop (versión optimizada para 64-bits).

Navegador

Opciones de presentación ▾

- ODBC (dsn=hive) [1]
- HIVE [4]
 - db_cepreami
 - db_cepreami1 [10]
 - cur_reporte_carreras
 - cur_reporte_colegios
 - dim_candidato
 - dim_carrera
 - dim_geografia
 - dim_institucion
 - dim_modalidad
 - dim_tiempo
 - fact_admision**
 - raw_postulantes
 - default
 - foodmart

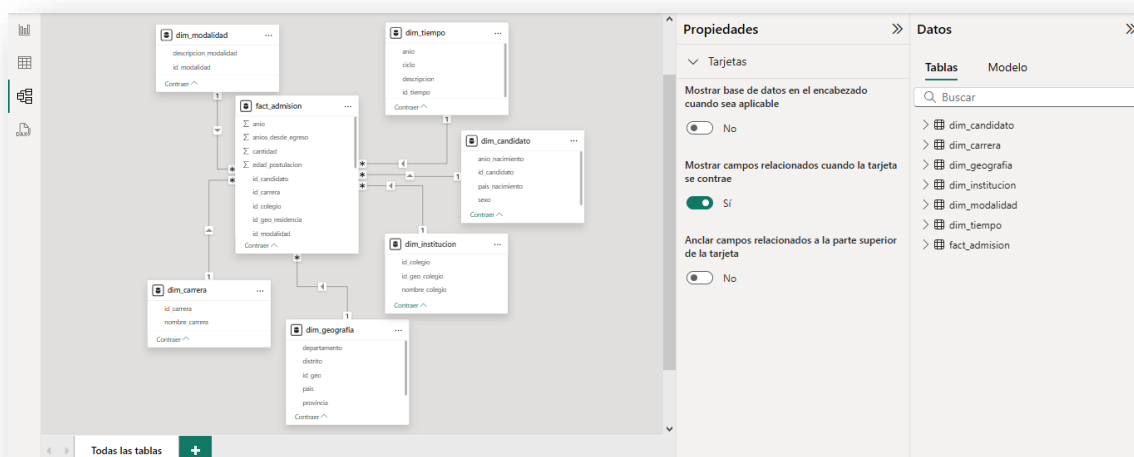
fact_admision

id_modalidad	id_geo_residencia	edad_postulacion	ani
1085	3 LIMA-LIMA-RIMAC		20
762	3 PUNO-SAN ROMAN-JULIACA		44
1581	3 LIMA-LIMA-INDEPENDENCIA		40
3794	3 LIMA-LIMA-VILLA EL SALVADOR		37
4523	2 SAN MARTIN-SAN MARTIN-TARAPOTO		35
436	3 LIMA-LIMA-VILLA EL SALVADOR		35
3279	3 CUSCO-CANCHIS-CHECACUPE		33
3392	3 LIMA-LIMA-LURIGANCHO		30
278	3 LIMA-LIMA-SANTA ANITA		30
4516	1 LIMA-LIMA-PUENTE PIEDRA		18
3367	3 LIMA-LIMA-COMAS		18
1608	3 HUANUCO-HUANUCO-SANTA MARIA DEL VAL		18
58	3 LIMA-LIMA-LURIGANCHO		18
671	1 AMAZONAS-CHACHAPOYAS-CHACHAPOYAS		18
2467	3 CALLAO-CALLAO-CALLAO		17
2425	3 ANCASH-HUARI-HUARI		18
263	3 CALLAO-CALLAO-BELLAVISTA		18
2237	3 LIMA-LIMA-LIMA		18
2711	3 AMAZONAS-UTCUBAMBA-LONYA GRANDE		18
1571	3 LIMA-LIMA-LIMA		18
791	3 PUNO-SAN ROMAN-JULIACA		18
657	3 PUNO-SAN ROMAN-JULIACA		18
791	3 AMAZONAS-CHACHAPOYAS-CHACHAPOYAS		18

Seleccionar tablas relacionadas Cargar Transformar datos Cancelar

Evidencia 1: Conexión de entorno Hive con Power BI Local mediante ODBC Driver

Descripción: Se descargó el ODBC Driver para conectar el Apache Hive con el Power BI, para la carga de tablas en el visualizador, se puede ver que la conexión es limpia y permite visualizar las tablas almacenadas en la base de datos usada.

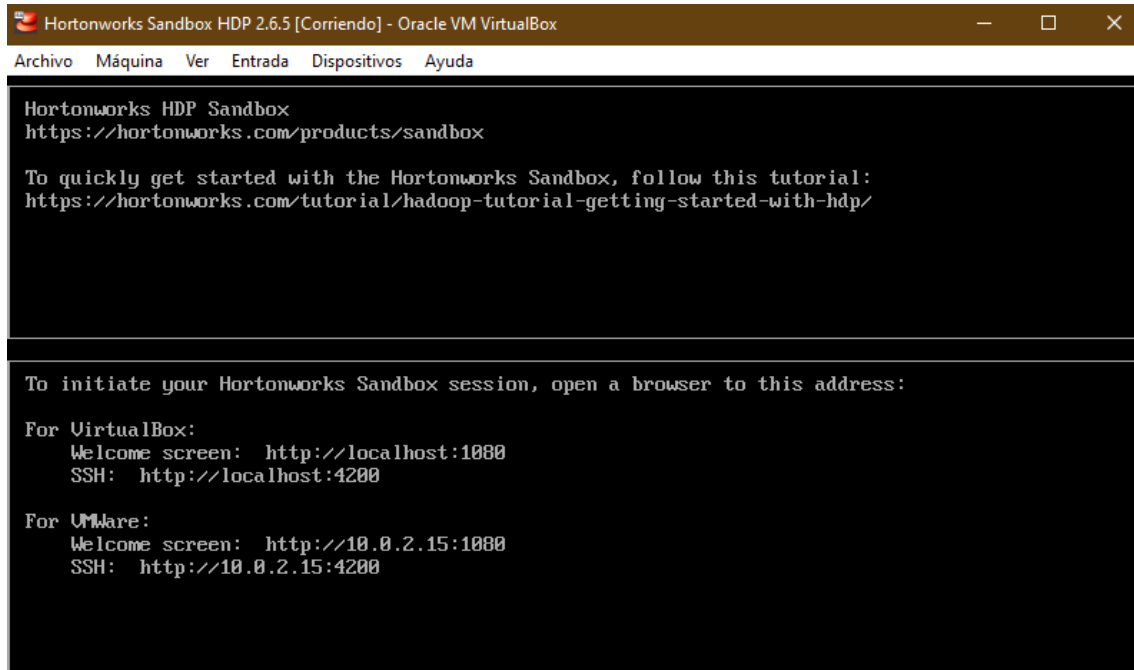


Evidencia 2: Conexión con el Modelo de Datos

Descripción: Diagrama del modelo semántico desplegado localmente, evidenciando las relaciones activas entre la tabla de hechos (fact_admision) y las dimensiones, validando la integridad del Esquema Estrella.

10. REPRODUCIBILIDAD DE LA SOLUCIÓN

Primero vamos al Oracle Virtual Box y encendemos la máquina virtual



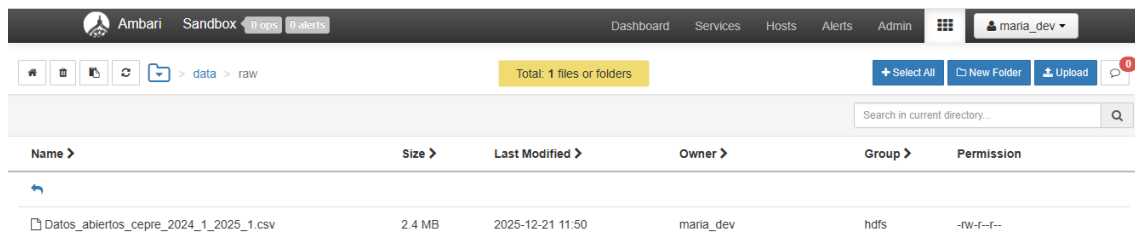
Luego vamos a la dirección y entramos a web Shell client, eso servirá para crear la estructura de carpetas



Usamos comandos como mkdir para la creación de carpetas.

```
sandbox-hdp login: root
root@sandbox-hdp.hortonworks.com's password:
Last login: Sun Dec 21 16:42:43 2025 from 172.18.0.2
[root@sandbox-hdp ~]#
```

1. Definir la ruta base para facilitar el proceso export
RUTA_BASE=/user/alumno/Ex_Sustitutorio/Cabana_Cazani_20212541J # 2. Crear las carpetas principales (Scripts y Notebooks) hdfs dfs -mkdir -p \$RUTA_BASE/scripts hdfs dfs -mkdir -p \$RUTA_BASE/notebooks # 3. Crear la estructura de DATA (Raw, Processed y Curated) hdfs dfs -mkdir -p \$RUTA_BASE/data/raw hdfs dfs -mkdir -p \$RUTA_BASE/data/processed hdfs dfs -mkdir -p \$RUTA_BASE/data/curated # 4. Crear las subcarpetas del Modelo Estrella dentro de PROCESSED hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/fact_admision hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_carrera hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_modalidad hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_tiempo hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_geografia hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_candidato hdfs dfs -mkdir -p \$RUTA_BASE/data/processed/dim_institucion # 5. Crear las subcarpetas de Reportes (KPIs) dentro de CURATED hdfs dfs -mkdir -p \$RUTA_BASE/data/curated/cur_reporte_carreras hdfs dfs -mkdir -p \$RUTA_BASE/data/curated/cur_reporte_colegios



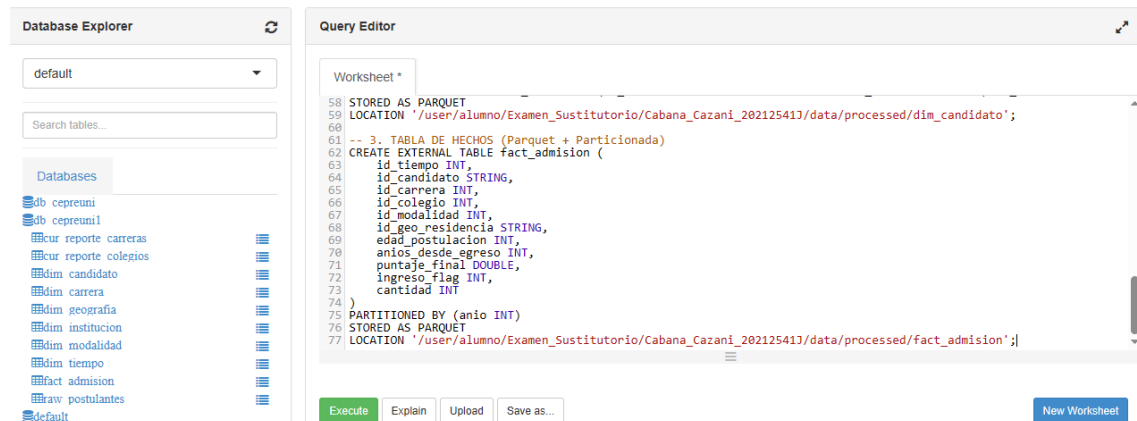
Name >	Size >	Last Modified >	Owner >	Group >	Permission
 Datos_abiertos_cepre_2024_1_2025_1.csv	2.4 MB	2025-12-21 11:50	maria_dev	hdfs	-rw-r--r--

Luego vamos al servicio de Hive para la creación de las tablas, ejecutamos lo siguiente, antes debemos crear nuestra base de datos con `create database db_cepreuni1`, en el texto, está el código, expandir para ver el código completo, de igual manera se encuentra el código en el repositorio.

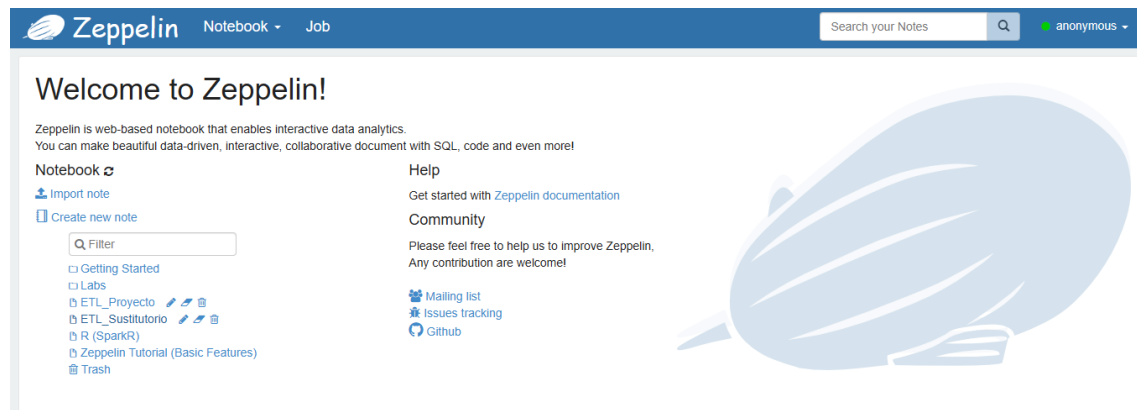
Link: <https://github.com/SI807-FIIS->

UNI/SI807_Cloud_BI_2025/blob/ExSusti_Cabana_Cazani_20212541J/Examen_Sustitutorio/Cabana_Cazani_20212541J/sql/Carga_tablas_Hive.sql

Usamos ese archivo para la carga SQL de las tablas de dimensiones y la tabla de hechos



Luego de crear las tablas nos vamos a usar Spark mediante Zeppelin



Creamos un nuevo notebook y ejecutamos para la limpieza

https://github.com/SI807-FIIS-UNI/SI807_Cloud_BI_2025/blob/ExSusti_Cabana_Cazani_20212541J/Examen_Sustitutorio/Cabana_Cazani_20212541J/etl/Lectura_Limpieza_bloque1.py

https://github.com/SI807-FIIS-UNI/SI807_Cloud_BI_2025/blob/ExSusti_Cabana_Cazani_20212541J/Examen_Sustitutorio/Cabana_Cazani_20212541J/etl/Carga_de_dimensiones_bloque2.py

https://github.com/SI807-FIIS-UNI/SI807_Cloud_BI_2025/blob/ExSusti_Cabana_Cazani_20212541J/Examen_Sustitutorio/Cabana_Cazani_20212541J/etl/Carga_final_hechos_bloque3.py

Ejecutamos en ese orden, primero la limpieza, luego la carga de dimensiones y finalmente la carga de hechos

```

--- BLOQUE 1: LECTURA Y LIMPieza ---
✓ Bloque 1 Completado. Registros listos: 8470

--- BLOQUE 2: CARGA DE DIMENSIONES ---
-> Procesando Carrera...
-> Procesando Modalidad...
-> Procesando Tiempo...
-> Procesando Geografía...
-> Procesando Institución...
-> Procesando Candidato...
✓ Bloque 2 Completado: Dimensiones cargadas.

--- BLOQUE 3: CARGA FINAL DE HECHOS ---
Escribiendo datos en fact_admision...
🔥 ¡PROYECTO TERMINADO! Base de datos db_cepreunil poblada.

```

Luego vamos a revisar en el File View, para ver mis carpetas con los archivos en formato parquet

Ambari

Sandbox

0 apps

0 alerts

Dashboard

Services

Hosts

Alerts

Admin

maria_dev

> processed > dim_carrera

Total: 2 files or folders

+ Select All

New Folder

Upload

0

Search in current directory...

Q

Name >	Size >	Last Modified >	Owner >	Group >	Permission
↶					
hdfs_SUCCESS	0.1 kB	2025-12-21 13:59	zeppelin	hdfs	-rw-r--r--
hdfs_part-00000-12e0d5ad-dff9-4d97-a5e6-0a...	1.2 kB	2025-12-21 13:59	zeppelin	hdfs	-rw-r--r--

> fact_admission > anio=2024

Total: 1 files or folders

+ Select All

New Folder

Upload

0

Search in current directory...

Q

Name >	Size >	Last Modified >	Owner >	Group >	Permission
↶					
hdfs_part-00000-ed7ecfd8-6ac0-475a-a94e-a6...	649.8 kB	2025-12-21 14:01	zeppelin	hdfs	-rwxr-xr-x

> fact_admission > anio=2025

Total: 1 files or folders

+ Select All

New Folder

Upload

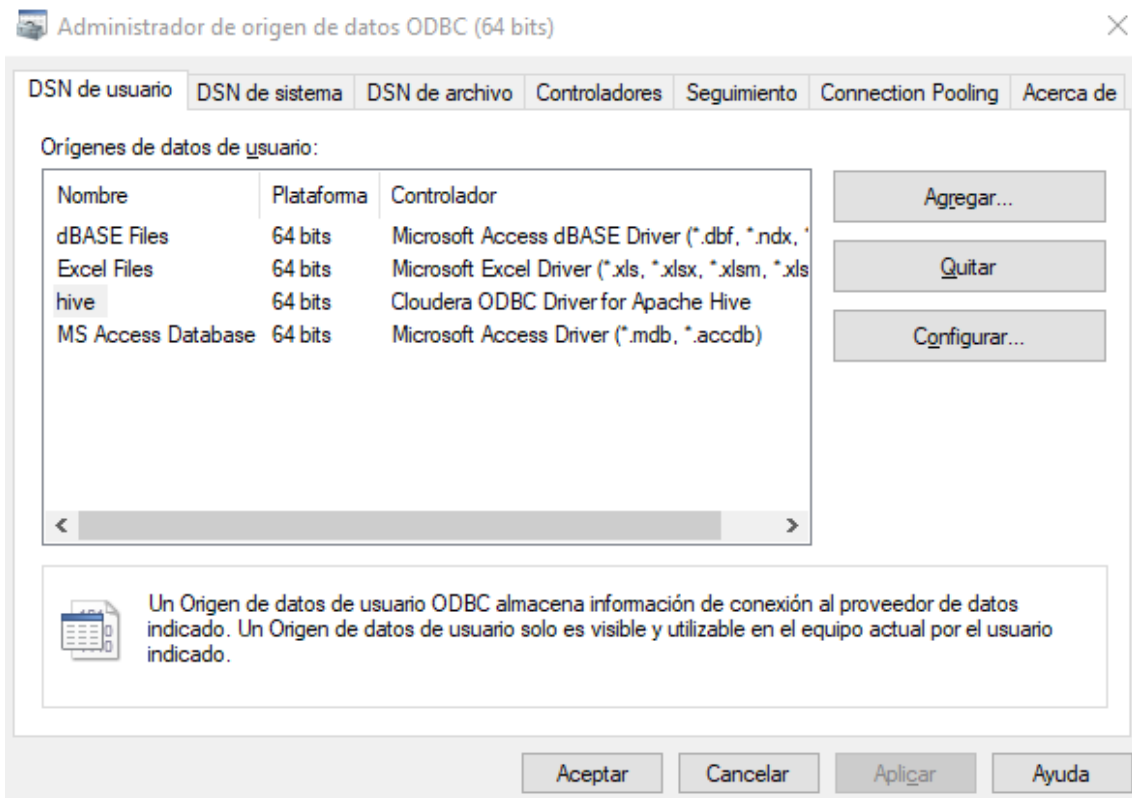
0

Search in current directory...

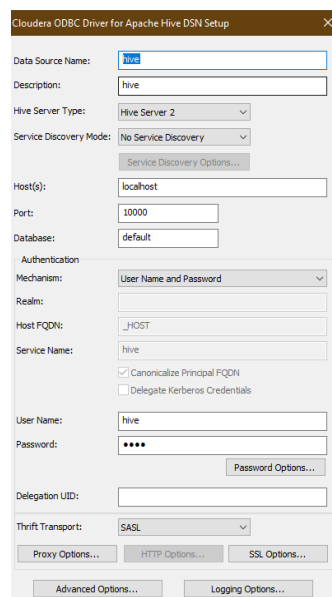
Q

Name >	Size >	Last Modified >	Owner >	Group >	Permission
↶					
hdfs_part-00000-ed7ecfd8-6ac0-475a-a94e-a6...	363.5 kB	2025-12-21 14:01	zeppelin	hdfs	-rwxr-xr-x

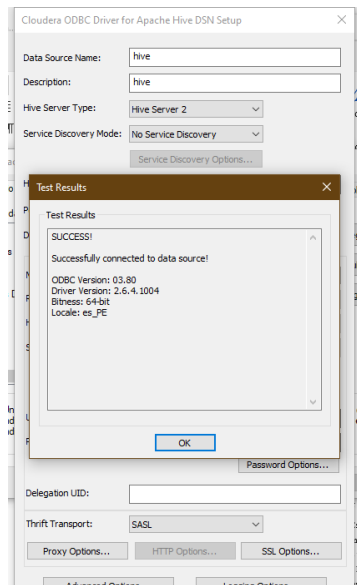
Revisando nos damos cuenta de que se realizó la carga en formato parquet, en todas las tablas, luego procedemos a conectar nuestro servicio con Power BI para elaborar los dashboards, para eso debemos descargar un driver ODBC para Apache Hive



Ese Hive previamente se creó con condiciones establecidas



Eso nos indica que se logró la conexión y es correcta la configuración



Ahora procedemos a abrir el Power BI y a obtener datos, escribimos ODBC

De ODBC

Nombre de origen de datos (DSN)

hive

► Opciones avanzadas

Aceptar

Cancelar

Luego nos puede pedir una contraseña, pero es la contraseña que se estableció para la configuración, en mi caso use una contraseña corta.

Luego de darle aceptar nos debería conectar con el Hive y mostrarnos las tablas de la base de datos db_ceprenil

Navegador

Opciones de presentación ▾

ODBC (dsn=hive) [1]

HIVE [4]

db_cepuni

db_cepuni1 [10]

cur_reporte_carreras

cur_reporte_colegios

dim_candidato

dim_carrera

dim_geografia

dim_institucion

dim_modalidad

dim_tiempo

fact_admision

raw_postulantes

default

foodmart

fact_admision

id_tiempo

id_candidato

id_carr

20241	BE11517F469351A6FA152F8AD70F9707812804B0112D18EABE5E8FCBAD	
20241	5519D203EC72BC2D14DEB410AFEB66E22C5E135A9232AB152408DD05C	
20241	FB5D70645AC934046646CDD8BD162714E324A8E444E5EE0E64E3C41FC3	
20241	063276C4ED0A1A0115AC54FACFD30F3DA43DD686E311B7DB32BE79FE9I	
20241	49275FD4C1059A9D9EA42C7C8FA4C86F1357E68F4287798256FAC4B75E	
20241	D8A9FCA90623AED486A9729702F7C72A1578284508D24871F744928D44	
20241	915CD32D5E6781E16FAB6D833CB483D538DE32175618D8BECABF4F29AI	
20241	AE1C391A973804E2AAE891DAD46D7B56A784223D2285F125D7C863D59	
20241	F9B046C1A4F557E49AA8E6FF73B5AD18A4EFACB80B02EF2BFCCDC3B40	
20241	4351D9F67C83DBA026D5D8AD233F81121FD2191AD8E836B53DF2FACB9	
20241	5350DA93D96266950C0727A4292817A3A51C93C6FD06A862501E6FAB6C	
20241	F71FD1B48C05FA2F183A66A881544D77125B458D86AFBB826E2133DB3C	
20241	6FB107F9CC90585B44CAF6233BF1B9FFB2701634DF5BF60887720F2B9	
20241	1FAD382EFB89727C55BEA775F51378C93CEC9F19D26586965729851B66E	
20241	F4CF9D85456F01D4355582FB18282C11775C13F552F3760B9CBD4BFEE81	
20241	6A71A227E3AF223835CEB1C9346CBF1769506A1DF0A0BCC45F0D0DC23E	
20241	4648DOC1D501E1F2518769C0E2DF7B24299E90D1AD88482655E0864BC7	
20241	433A1DC7E7B0434666ABC067CEABCF7AD68BB7C087DB34F1EC69E4804F	
20241	AE522D30B7B345EB4214C70F32224955038FDB43262E15AF6E4E210229E	
20241	3D748415C93A18E6D9D46CEC7EF44C251FD034424CE592BCE54552F7A3	
20241	5C0C633F1C3D72664FC890CA864C227A23743B97E07A1A5A1598959C57	
20241	59E05E9DD19FA31D5877A5BCEFF487464A78AA7601A1ECCF5E72E4AC68	
20241	C15822BDBCE2286A48CB4059925961D92DAC1DEC831943B8A6574150E1	

Seleccionar tablas relacionadas

Cargar

Transformar datos

Cancelar

En la imagen se aprecia que efectivamente se conectó y podemos cargar las tablas y con ello elaborar los dashboards.

Resultado final luego de elaborar los dashboards.

Reporte Académico - Admisión UNI

Carrera

Todas

sexo

Todas

departamento

Todas

Total Ingresantes por Colegio

Colegio	Total Ingresantes
SACO OLIVEROS	113.0
TRILCE	87.0
PAMER	20.0
PROLOG	13.0
CLAUDIO GALE...	9.0
TECHNOLOGY S...	9.0
EL NAZARENO	7.0
TRILCE LOS OL...	7.0
ALFRED NOBEL	5.0

Total Ingresantes por sexo

sexo	Total Ingresantes
MASCULINO	1.3 mil (80.47%)
FEMENINO	0.3 mil (19.53%)

Carrera

Nota Máxima

Carrera	Nota Máxima
ARQUITECTURA	16.32
CIENCIA DE LA COMPUTACIÓN	14.62
FÍSICA	15.12
INGENIERÍA AEROSPAIAL	16.31
INGENIERÍA AMBIENTAL	13.44
INGENIERÍA CIVIL	16.82
INGENIERÍA DE CIBERSEGURIDAD	15.77
INGENIERÍA DE HIGIENE Y SEGURIDAD	13.68
INDUSTRIAL	
INGENIERÍA DE MINAS	15.32
INGENIERÍA DE PETROLEO Y GAS	13.75
NATURAL	
Total	17.34

Tasa de Ingreso %

18.50 %

Nota Máxima

17.34

% Ausentismo

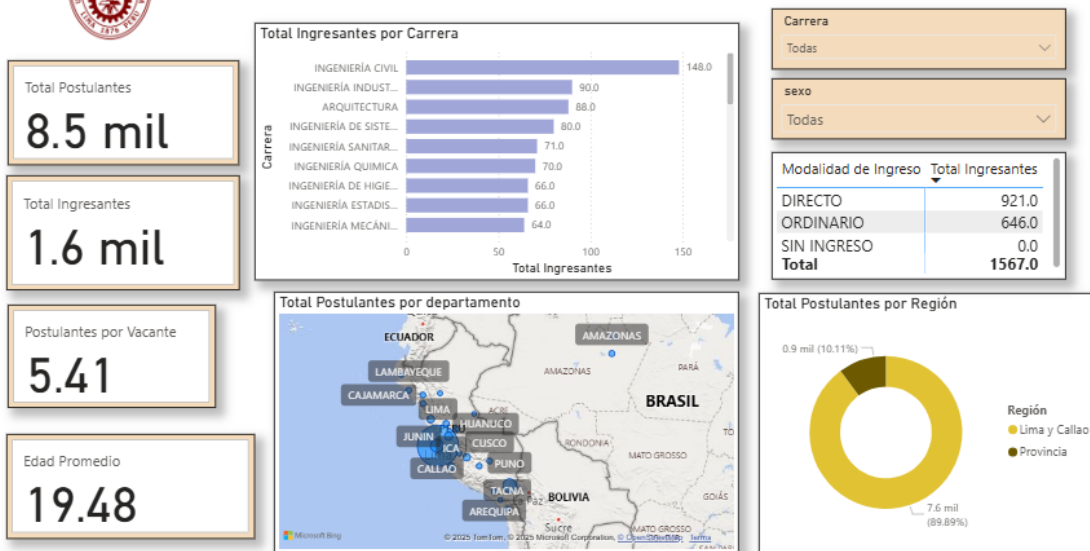
1.49 %

Total Ingresantes por departamento

departamento	Total Ingresantes
LIMA	1.4 mil
CALLAO	0.1 mil
PUNO	0.1 mil
JUNIN	0.0 mil
ICA	0.0 mil



Reporte de Gestión y Demanda - Admisión UNI



11. RESULTADOS Y ANÁLISIS

Tras la implementación de la arquitectura de Big Data y la construcción de los tableros de control en Power BI, se procedió a la explotación de la información histórica. A continuación, se detallan los hallazgos más relevantes obtenidos del análisis de los datos procesados, divididos por dimensiones estratégicas de negocio.

11.1. Análisis de Gestión y Demanda (Operativo)

El análisis del Dashboard de Gestión permite evaluar la eficiencia de la convocatoria, el comportamiento del mercado y la cobertura de vacantes.

- Alta Competitividad del Mercado:** El proceso de admisión analizado registró un volumen total de **8.5 mil postulantes**, de los cuales **1.6 mil lograron obtener una vacante**. Esto genera un indicador de Competitividad (*Postulantes por Vacante*) de **5.41**; es decir, por cada vacante disponible en la universidad, compiten más de 5 estudiantes. Este ratio confirma que la institución mantiene un posicionamiento sólido y una alta demanda en el mercado educativo.
- Centralización Geográfica de la Demanda:** El análisis de procedencia revela una marcada centralización en la capital. El **89.89%** de los postulantes reside en **Lima y Callao** (aprox. 7.6 mil), mientras que solo el **10.11%** proviene de provincias. Este hallazgo sugiere una oportunidad estratégica para descentralizar los procesos de captación y ampliar la oferta hacia el interior del país.

3. **Preferencias por Especialidad:** Las carreras de ingeniería tradicional dominan la preferencia de los ingresantes. **Ingeniería Civil** se consolida como la especialidad con mayor ingreso (**148 vacantes cubiertas**), seguida por **Ingeniería Industrial (90)** y **Arquitectura (88)**. Estas cifras permiten a la dirección académica planificar la distribución de recursos y aulas en función de la demanda real.
4. **Perfil Etario del Postulante:** La **edad promedio** de los postulantes se sitúa en **19.48 años**. Considerando que la educación secundaria culmina típicamente a los 16 o 17 años, este dato evidencia que el postulante promedio invierte entre 2 y 3 años en preparación preuniversitaria antes de lograr el ingreso o durante sus intentos de admisión.

11.2. Análisis Académico (Calidad y Perfil)

El Dashboard Académico permitió profundizar en la selectividad del proceso y caracterizar el perfil de los ingresantes exitosos.

1. **Alta Selectividad (Tasa de Ingreso):** Se calculó una **Tasa de Ingreso general del 18.50%**. Este indicador valida el rigor académico del examen de admisión, demostrando que menos de la quinta parte de los inscriptos logra superar las evaluaciones satisfactoriamente, lo que garantiza un alto estándar en el alumnado admitido.
2. **Excelencia Académica y Notas:** La nota máxima registrada en el Cómputo General fue de **17.34** (en escala vigesimal). Al realizar un desglose por carrera, se observa que los ingresantes a **Ingeniería Civil** y **Aeroespacial** poseen los promedios más altos, con máximos referenciales superiores a 16.00, lo que implica que dichas carreras captan al talento con mejor rendimiento académico.
3. **Brecha de Género en STEM:** Se evidencia una disparidad significativa en la distribución demográfica. El **80.47%** de los ingresantes corresponde al género **Masculino**, frente a un **19.53%** de género **Femenino**. Este hallazgo cuantitativo pone de manifiesto la necesidad de fortalecer políticas institucionales que fomenten la participación de mujeres en carreras de ingeniería y ciencias.
4. **Procedencia Escolar (Instituciones):** El ranking de colegios de procedencia muestra una hegemonía de instituciones privadas con sistemas preuniversitarios integrados. El colegio **"Saco Oliveros"** lidera la lista con **113 ingresantes**, seguido por **"Trilce" (87)** y **"Pamer" (20)**. Estos datos permiten identificar qué instituciones de Educación Básica Regular (EBR) son las principales "proveedoras" de talento para la universidad.

5. **Ausentismo:** El indicador de **Ausentismo fue de 1.49%**, una cifra marginal que refleja el alto compromiso de los postulantes inscritos, minimizando las pérdidas operativas por inasistencia el día de la prueba.

11.3. Síntesis de Resultados

La solución de Inteligencia de Negocios ha permitido transformar datos dispersos en conocimiento accionable. Los resultados demuestran que el proceso de admisión es **altamente selectivo, centralizado geográficamente y con una clara predominancia masculina en el ingreso**. Estas métricas, ahora automatizadas, facultan a las autoridades para tomar decisiones basadas en evidencia (*Data-Driven*) para futuros procesos de admisión.

12. CONCLUSIONES

Como resultado del diseño, implementación y análisis de la solución de Inteligencia de Negocios y Big Data para el proceso de admisión, se establecen las siguientes conclusiones:

Se concluye que la implementación de una arquitectura escalable basada en el ecosistema **Hadoop** fue exitosa y necesaria. El uso de **Apache Spark** para el proceso ETL permitió gestionar eficazmente la limpieza y deduplicación de registros históricos, reduciendo la inconsistencia de los datos originales y garantizando una "fuente única de verdad" confiable para la toma de decisiones.

El despliegue del Data Warehouse en **Apache Hive** bajo un esquema dimensional (Estrella) mejoró drásticamente los tiempos de respuesta para consultas analíticas. Se logró transitar de un modelo operativo basado en reportes manuales y archivos dispersos, a un modelo analítico centralizado que permite la exploración dinámica de años de historia académica en segundos.

El análisis cuantitativo corroboró el alto nivel de exigencia del proceso de admisión, evidenciado por una **Tasa de Ingreso del 18.50%** y un índice de competitividad superior a **5 postulantes por vacante**. Estos indicadores confirman que la institución mantiene su prestigio basándose en un filtro académico riguroso, donde solo los postulantes con mayor preparación logran el acceso.

Los tableros de control revelaron desequilibrios estructurales en la población de postulantes. La alta concentración de la demanda en **Lima y Callao (aprox. 90%)** y la marcada brecha de género (**80% hombres vs. 20% mujeres**) concluyen que existen segmentos de mercado desatendidos (provincias y población femenina) que representan una oportunidad estratégica de crecimiento e inclusión para la universidad.

La integración con **Microsoft Power BI** demostró ser una herramienta fundamental para la gestión. La capacidad de filtrar la información por colegio de procedencia, carrera y modalidad

en tiempo real faculta a las autoridades para diseñar estrategias de marketing más precisas (enfocadas en colegios específicos como Saco Oliveros o Trilce) y optimizar la planificación de vacantes basada en la demanda real y no en estimaciones empíricas.

13. LIMITACIONES Y MEJORAS FUTURAS

13.1. Limitaciones de la Solución Actual

A pesar de que el sistema cumple con los objetivos de análisis descriptivo, se identificaron las siguientes restricciones técnicas y operativas durante el desarrollo del proyecto:

1. **Dependencia de Procesos Manuales (Batch):** La arquitectura actual opera bajo un esquema de procesamiento por lotes (*Batch Processing*), donde la ingesta de datos depende de la extracción manual de archivos planos (CSV) desde los sistemas transaccionales heredados. Esto implica que la información visualizada en los tableros no es en tiempo real, existiendo una latencia entre el cierre del proceso de admisión y la actualización de los indicadores.
2. **Infraestructura Local (On-Premise):** El clúster de Big Data (Hadoop/Spark) se encuentra desplegado en un entorno virtualizado local. Esta configuración presenta limitaciones físicas de hardware (memoria RAM y núcleos de CPU), lo que restringe la capacidad de procesamiento paralelo si el volumen de datos creciera exponencialmente (escalabilidad vertical limitada).
3. **Alcance Limitado al Proceso de Admisión:** El modelo de datos actual finaliza en el momento en que el postulante ingresa. No existe integración con los sistemas académicos posteriores de la universidad (como ORCE u OCR), lo que impide correlacionar el puntaje de ingreso con el desempeño académico real del alumno durante su carrera universitaria.

13.2. Mejoras Futuras y Trabajo Futuro

Para escalar la solución y aumentar su valor estratégico, se proponen las siguientes líneas de evolución:

1. **Implementación de Analítica Predictiva (Machine Learning):** Aprovechar las librerías de **Spark MLlib** para entrenar modelos predictivos sobre la data histórica depurada. Se propone desarrollar un modelo de clasificación para predecir la probabilidad de ingreso de un alumno basándose en sus simulacros previos y colegio de procedencia, así como modelos de regresión para estimar la demanda futura de vacantes por carrera.

2. **Migración a Arquitectura Cloud:** Migrar la solución a un entorno de nube pública (como AWS EMR, Google Dataproc o Azure Synapse). Esto permitiría desacoplar el almacenamiento del cómputo, habilitando una escalabilidad elástica y reduciendo los costos de mantenimiento de infraestructura física.
3. **Automatización y Orquestación del Flujo ETL:** Implementar herramientas de orquestación de flujos de trabajo como **Apache Airflow**. Esto permitiría programar la ejecución automática de los scripts de limpieza y carga (pipelines) de forma periódica, eliminando la intervención manual y asegurando la frescura de los datos.
4. **Integración de Datos No Estructurados:** Enriquecer el análisis incorporando fuentes de datos no estructuradas, como comentarios en redes sociales o encuestas de satisfacción abiertas. Esto permitiría realizar un "Análisis de Sentimiento" para entender la percepción cualitativa de los postulantes sobre el servicio brindado por CEPRE-UNI.

14. REFERENCIAS

- Plataforma Nacional de Datos Abiertos. (2024). *Alumnos del Centro Pre Universitario de la Universidad Nacional de Ingeniería (CEPRE-UNI)* [Dataset]. Gobierno del Perú. Recuperado de: <https://www.datosabiertos.gob.pe/dataset/alumnos-del-centro-pre-universitario-de-la-universidad-nacional-de-ingenier%C3%ADa-del-2024-al-3>
- Apache Software Foundation. (2024). *Apache Spark Documentation: PySpark Overview*. Recuperado de: <https://spark.apache.org/docs/latest/api/python/>
- Microsoft. (2024). *Documentación de Power BI: Conceptos básicos para diseñadores*. Microsoft Learn. Recuperado de: <https://learn.microsoft.com/es-es/power-bi/>

ANEXOS

Visualización de Dashboards en Github

https://github.com/SI807-FIIS-UNI/SI807_Cloud_BI_2025/tree/ExSusti_Cabana_Cazani_20212541J/Examen_Sustitutorio/Cabana_Cazani_20212541J/dashboards